

Question 2: Bias Gradients, Parameter Sharing, and Activation Effects

Given Network and Notation

We are given the following forward pass for one training example (x, y) :

$$\begin{aligned}h_1 &= g(Wx + b) \\ \hat{y} &= g_1(Uh_1 + b) \\ L &= \ell(\hat{y}, y)\end{aligned}$$

where:

- $x \in \mathbb{R}^d$ is the input feature vector,
- $y \in \mathbb{R}^k$ is the target vector,
- $W \in \mathbb{R}^{m \times d}$ and $U \in \mathbb{R}^{k \times m}$ are weight matrices,
- $b \in \mathbb{R}^m$ is a single shared bias vector used in both affine transformations,
- $g(\cdot)$ is the hidden activation function (element-wise),
- $g_1(\cdot)$ is the output activation function (element-wise),
- $\ell(\cdot, \cdot)$ is a differentiable loss.

For simplicity, the question assumes $m = k$, so the same bias vector $b \in \mathbb{R}^m$ can be added in both layers.

Part 1: Derive the Shared Bias Gradient $\frac{\partial L}{\partial b}$

To make the chain rule easy to follow, define intermediate variables:

$$\begin{aligned}z_1 &= Wx + b \\ h_1 &= g(z_1) \\ z_2 &= Uh_1 + b \\ \hat{y} &= g_1(z_2) \\ L &= \ell(\hat{y}, y)\end{aligned}$$

Now notice the key point: the same b appears in **two places**:

- in $z_1 = Wx + b$ (first affine transform),
- in $z_2 = Uh_1 + b$ (second affine transform).

So b influences the loss through **two computational paths**. Because of that, the final gradient $\frac{\partial L}{\partial b}$ must include contributions from both paths.

Step 1: Start from the output side

By chain rule,

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial z_2} \frac{\partial z_2}{\partial b} + \frac{\partial L}{\partial z_1} \frac{\partial z_1}{\partial b}$$

We will compute each piece carefully.

Step 2: Compute $\frac{\partial L}{\partial z_2}$

We have $\hat{y} = g_1(z_2)$ and $L = \ell(\hat{y}, y)$, so:

$$\frac{\partial L}{\partial z_2} = \frac{\partial L}{\partial \hat{y}} \odot g'_1(z_2)$$

Define:

$$\delta_2 \triangleq \frac{\partial L}{\partial z_2}$$

So:

$$\delta_2 = \frac{\partial L}{\partial \hat{y}} \odot g'_1(z_2)$$

Step 3: Contribution of b from the second layer

Since $z_2 = Uh_1 + b$, the derivative of z_2 with respect to b is:

$$\frac{\partial z_2}{\partial b} = I$$

Therefore the gradient contribution from the second occurrence of b is:

$$\left(\frac{\partial L}{\partial z_2} \right) \left(\frac{\partial z_2}{\partial b} \right) = \delta_2$$

Step 4: Compute $\frac{\partial L}{\partial z_1}$

We need to backpropagate from z_2 to z_1 .

Since $z_2 = Uh_1 + b$,

$$\frac{\partial L}{\partial h_1} = U^T \frac{\partial L}{\partial z_2} = U^T \delta_2$$

Now because $h_1 = g(z_1)$, we apply the activation derivative:

$$\frac{\partial L}{\partial z_1} = \left(\frac{\partial L}{\partial h_1} \right) \odot g'(z_1) = (U^T \delta_2) \odot g'(z_1)$$

Define:

$$\delta_1 \triangleq \frac{\partial L}{\partial z_1}$$

So:

$$\delta_1 = (U^T \delta_2) \odot g'(z_1)$$

Step 5: Contribution of b from the first layer

Since $z_1 = Wx + b$, again:

$$\frac{\partial z_1}{\partial b} = I$$

So the gradient contribution from the first occurrence of b is:

$$\left(\frac{\partial L}{\partial z_1} \right) \left(\frac{\partial z_1}{\partial b} \right) = \delta_1$$

Final Answer: Shared Bias Gradient

Now add both contributions (because b appears twice in the computation graph):

$$\boxed{\frac{\partial L}{\partial b} = \delta_2 + \delta_1}$$

where

$$\delta_2 = \frac{\partial L}{\partial \hat{y}} \odot g'_1(z_2), \quad \delta_1 = (U^T \delta_2) \odot g'(z_1)$$

This shows clearly how the same parameter b receives gradients from multiple paths, and the final gradient is the sum of those path contributions.

Part 2: Optimization and Convergence Effect (Shared vs Independent Bias)

We compare two models:

- **Shared-bias model:** the same b is used in both layers.
- **Independent-bias model:** each layer has its own bias (b_1 and b_2).

Do we expect bias sharing to affect convergence speed or stability?

Yes, it can affect both speed and stability. The main reasons are:

- **Parameter coupling:** In the shared case, a single update to b changes both layers at once. That means the model has less freedom to shift the hidden layer and output layer independently.
- **Fewer degrees of freedom:** Independent biases have more parameters (b_1 and b_2), so they can fit offsets separately. This can make optimization easier in some cases because the model can adjust each layer without forcing a compromise.
- **Gradient scaling (two paths):** In the shared case the gradient is $\delta_2 + \delta_1$. This can lead to larger updates (compared to a single-layer bias gradient), which may speed up learning in early steps, but can also make updates noisier or less stable depending on the directions of δ_1 and δ_2 .
- **Possible gradient conflict:** If δ_1 and δ_2 point in different directions, their sum may partially cancel, meaning the shared bias update becomes weaker or inconsistent across iterations.

So overall, bias sharing can change convergence because it reduces flexibility and mixes gradient signals coming from different layers.